SERVI-X Modelado de la Solución

(Actualizada con Especificación de Requisitos, Propiedad Intelectual, Contrato de Licencia, Trazabilidad y Pruebas de Integración)

Jose Alvarez Marin

Facultad de Ingeniería, Corporación Universitaria de Sabaneta
Integración de Aplicaciones y Sistemas

Dr. Mauricio Fabian Agudelo
17 de Octubre de 2026

Resumen

Este documento presenta el modelado técnico detallado para el módulo SERVI-X, responsable de la gestión integral de Peticiones, Quejas, Reclamos y Sugerencias (PQRS) dentro del ecosistema de microservicios INTEGRAX. A través de un enfoque de diseño riguroso que abarca el Modelo de Datos (Entidad-Relación) y el modelado UML (Casos de Uso, Clases y Secuencia), se define la estructura lógica, el almacenamiento y el comportamiento dinámico del sistema. El diseño propuesto garantiza el aislamiento de datos propio de la arquitectura del proyecto, prioriza la trazabilidad inmutable de las actuaciones (auditoría) y establece los mecanismos de integración asíncrona y síncrona vía API REST con los módulos externos CRM-X, SALES-X y LOGISTI-X.

Nota sobre esta actualización: respecto a la versión anterior (28 de agosto de 2026), este documento incorpora la Especificación de Requisitos de Software bajo lineamientos IEEE 830 (Introducción, Propósito, Alcance, Restricciones, Interfaces, Reglas de Negocio y Criterios de Aceptación), amplía el Diccionario de Datos con la columna de longitud de cada campo, y añade cuatro capítulos nuevos: Matriz de Propiedad Intelectual, Contrato de Licencia de Software, Matriz de Trazabilidad y Pruebas de Integración. El modelado UML, el modelo entidad-relación y el diagrama de secuencia ya desarrollados se conservan sin alteraciones de fondo.

Alcance vigente del módulo: en esta fase, SERVI-X opera de forma **aislada**. Las integraciones con los servicios externos del ecosistema (CRM-X, SALES-X y LOGISTI-X) están diseñadas en el modelo de datos mediante llaves foráneas lógicas (*cliente_id*, *pedido_id*, *despacho_id*), pero su consumo real vía HTTP/REST queda **planificado para una fase posterior** del cronograma de INTEGRAX; no se encuentran implementadas ni activas en la presente entrega.

Diagrama de Gantt y Metodología de Trabajo

El proyecto se gestiona con una **metodología híbrida**: el cronograma general de INTEGRAX fija hitos semanales de entrega obligatoria (viernes de cada semana, del 21 de agosto al 18 de septiembre de 2026 según el cronograma oficial del curso), mientras que el trabajo dentro de cada semana se organiza de forma ágil mediante ramas de feature en Git (*feature/** → *dev* → *main*), con aprobación explícita antes de cada integración. Esto permite entregar incrementos funcionales verificables en cada hito sin perder la trazabilidad de los cambios.

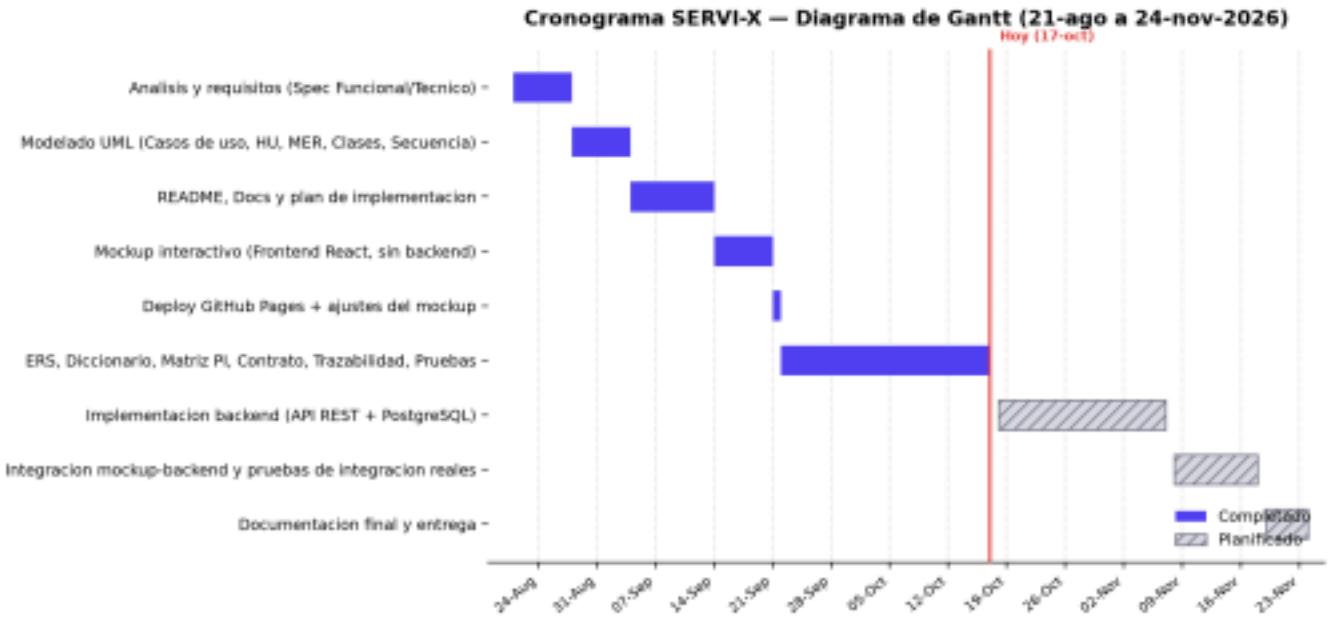


Figura 0. Diagrama de Gantt del proyecto SERVI-X - fases completadas y planificadas.

Las primeras seis fases (análisis, modelado UML, documentación de base y mockup interactivo desplegado en GitHub Pages) ya se completaron y están verificadas en el repositorio del proyecto. Las fases de implementación del backend, integración real con el mockup y pruebas de integración ejecutadas quedan planificadas para las siguientes semanas del curso, conforme al Plan de Implementación documentado en *Docs/Plan-Implementacion.md*.

1. Especificación de Requisitos de Software (ERS - IEEE 830)

1.1 Introducción

La presente Especificación de Requisitos de Software (ERS) documenta de manera formal las necesidades funcionales y no funcionales del módulo SERVI-X, siguiendo la estructura recomendada por el estándar IEEE 830. Su propósito es servir como contrato técnico entre el equipo desarrollador y los interesados del proyecto académico INTEGRAX, garantizando una base verificable para el diseño, la implementación y las pruebas del sistema.

1.2 Propósito

El propósito de este documento es especificar, sin ambigüedad, el comportamiento esperado del módulo SERVI-X: qué debe hacer el sistema (requisitos funcionales), bajo qué condiciones de calidad debe operar (requisitos no funcionales), y qué restricciones técnicas, reglas de negocio y criterios de aceptación enmarcan su desarrollo. Está dirigido al equipo desarrollador, al docente evaluador y, a futuro, a los equipos responsables de los demás módulos de INTEGRAX que

consumirán sus servicios.

1.3 Alcance

SERVI-X es el módulo encargado de centralizar, clasificar, asignar, atender, responder y cerrar las solicitudes de PQRS de los clientes de la plataforma INTEGRAX, garantizando trazabilidad inmutable de cada actuación.

Aclaración de alcance de la fase actual: el sistema se construye y valida como un **módulo aislado**, con su propia base de datos PostgreSQL independiente (sin tablas ni vistas compartidas con otros módulos). Las integraciones salientes hacia CRM-X, SALES-X y LOGISTI-X están **fuera del alcance funcional de esta entrega**: se documentan y se reserva el modelo de datos para ellas (campos cliente_id, pedido_id, despacho_id como llaves foráneas lógicas), pero su consumo real vía HTTP/REST queda planificado para una integración posterior dentro del cronograma general de INTEGRAX.

1.4 Descripción general

1.4.1 Perspectiva del producto: SERVI-X es un sistema web compuesto por un frontend en React (Vite) y una API REST en Node.js/TypeScript (Express o NestJS), respaldada por una base de datos relacional PostgreSQL. Es uno de los seis microproyectos independientes que conforman INTEGRAX y está diseñado bajo una Arquitectura Orientada a Servicios (SOA) para integrarse posteriormente con los módulos restantes.

1.4.2 Funciones principales: registro y autenticación de usuarios; radicación, clasificación, consulta y seguimiento de PQRS; asignación de responsables; actualización de estados con trazabilidad obligatoria; registro de respuestas oficiales; cierre controlado de casos; generación de reportes de gestión.

1.4.3 Usuarios del sistema:

- **Cliente:** radica solicitudes, consulta sus propias solicitudes y su historial, y visualiza las respuestas oficiales.
- **Agente de servicio:** clasifica, asigna, gestiona el estado, responde, cierra casos y genera reportes de gestión.

1.5 Requisitos Funcionales

Los requerimientos funcionales definen el comportamiento operativo, las reglas de negocio y las capacidades de interacción del módulo SERVI-X. Este conjunto de especificaciones detalla las acciones precisas que el sistema debe ejecutar para gestionar el ciclo de vida completo de una

PQRS, desde su radicación inicial hasta su cierre, asegurando la correcta integración transaccional con los actores y los sistemas externos del ecosistema INTEGRAX.

ID	Nombre del Requerimiento	Descripción Detallada
RF-01	Registrar PQRS	Permite a clientes o agentes crear una nueva solicitud en el sistema, generando un identificador único (UUID v4) e iniciando su ciclo de vida.
RF-02	Clasificar solicitudes	Facilita la categorización de la solicitud ingresada según su tipología exacta (Petición, Queja, Reclamo o Sugerencia) y nivel de urgencia.
RF-03	Asignar responsables	Permite vincular un agente operativo específico (responsable_id) a un caso abierto para gestionar su resolución.
RF-04	Consultar solicitudes	Habilita la visualización de un listado general de casos con sus respectivos estados actuales, aplicable tanto para clientes como para agentes.
RF-05	Actualizar estado de atención	Permite modificar la etapa del caso (ej. radicado, en revisión, resuelto), exigiendo justificación y registrando la transición en la bitácora.
RF-06	Registrar respuestas	Habilita la redacción e inserción de la respuesta formal que se brindará al cliente como solución a su solicitud.
RF-07	Cerrar casos	Finaliza el ciclo de vida de la PQRS, estampando la fecha de cierre y bloqueando el registro contra modificaciones futuras.
RF-08	Consultar historial de atención	Muestra la línea de tiempo inmutable de un caso, detallando fechas, horas, agentes involucrados y transiciones de estado extraídas de la tabla de auditoría.
RF-09	Buscar solicitudes por cliente	Función de soporte que permite filtrar y listar todas las solicitudes históricas y activas asociadas a un identificador de cliente específico.
RF-10	Generar reportes de gestión	Consolida métricas operativas (cantidad de quejas por estado, tiempos de respuesta, tipologías) consultando directamente la base de datos aislada.

RF-11	Consultar información asociada a pedidos y despachos	Permite a SERVI-X consumir APIs de módulos externos (SALES-X, LOGISTI-X) para validar la existencia de datos comerciales asociados a la queja. (Integración planificada para fase posterior; no activa en esta entrega.)

ID	Nombre del Requerimiento	Descripción Detallada
RF-12	Exponer servicios para integración empresarial	Provee endpoints para que otros módulos del ecosistema INTEGRAX u otras plataformas externas puedan radicar o consultar casos.

1.6 Requisitos No Funcionales

Los requerimientos no funcionales, entendidos en el diseño moderno como características arquitectónicas críticas, establecen las restricciones técnicas y los atributos de calidad que garantizan la viabilidad del sistema. En lugar de definir qué hace el software, estas métricas dictan cómo debe operar bajo condiciones de estrés, priorizando pilares fundamentales como la disponibilidad, el rendimiento, la trazabilidad inmutable y el cumplimiento estricto de la arquitectura de microservicios.

ID	Categoría	Descripción Detallada
RNF-01	Disponibilidad	El módulo SERVI-X debe garantizar un tiempo de operatividad mínimo del 95%, asegurando el acceso continuo al registro y consulta de casos.
RNF-02	Rendimiento	Toda petición estándar realizada al backend debe procesarse y retornar una respuesta al cliente en un tiempo inferior a 3 segundos (3000 ms).
RNF-03	Usabilidad	La interfaz de usuario debe ser intuitiva, facilitando la navegación sin requerir capacitación extensa, adaptándose a distintos tamaños de pantalla.
RNF-04	Seguridad	El sistema debe validar la identidad mediante tokens de autenticación y restringir el acceso a las funciones operativas según los roles (Cliente vs. Agente).

RNF-05	Auditabilidad	El diseño debe garantizar la persistencia de una bitácora inmutable (tabla Historial_Atencion) que impida la sobrescritura del pasado ante cualquier cambio.
RNF-06	Arquitectura	Toda la comunicación entre el frontend, SERVI-X y los demás módulos (CRM-X, SALES-X) debe realizarse estrictamente bajo los principios de la arquitectura REST.
RNF-07	Portabilidad	Las vistas de radicación y seguimiento de PQRS deben renderizarse y funcionar de manera óptima en los navegadores de dispositivos móviles y tablets.
RNF-08	Escalabilidad	La base de datos y los servicios deben soportar el incremento transaccional sin degradación, apoyándose en el uso de identificadores UUID v4 para redes distribuidas.

1.7 Restricciones

• Lenguaje y entorno de ejecución: Node.js 20+ con TypeScript.

• Framework web: Express.js (o NestJS como alternativa documentada).
• Frontend: React con Vite.

• Motor de base de datos: PostgreSQL 15+, exclusivo del módulo (sin tablas compartidas con otros microproyectos).

• Todas las llaves primarias y foráneas deben ser UUID v4; queda prohibido el uso de enteros autoincrementales.

• Nomenclatura de base de datos en snake_case; contratos de API en camelCase.

1.8 Interfaces

• **Interfaz de usuario:** aplicación web responsiva (portal Cliente y portal Agente), accesible desde navegador en escritorio y móvil.

• **Interfaz de hardware:** ninguna especial; el sistema corre sobre infraestructura de servidor virtualizada/en contenedores, sin dependencia de hardware dedicado.

• **Interfaz de software:** API REST en formato JSON como único canal de comunicación entre el frontend y el backend, y entre SERVI-X y los módulos externos (cuando se habilite la integración).

1.9 Reglas de Negocio

ID	Regla
RN-01	Todo identificador (caso, usuario, historial) debe ser UUID v4, nunca secuencial.
RN-02	SERVI-X opera sobre su propio repositorio de datos; debe seguir funcionando aunque otros módulos del ecosistema fallen.
RN-03	No se puede cerrar un caso si no existe al menos una respuesta oficial registrada previamente.
RN-04	Todo caso nuevo debe generar simultáneamente su primer registro en la bitácora de auditoría.

1.10 Criterios de Aceptación

• El sistema permite registrar, autenticar, radicar, asignar, actualizar, responder, cerrar y consultar PQRS cumpliendo los 10 casos de uso definidos.

• Ninguna operación de cierre se ejecuta sin respuesta oficial previa (validación de RN-03 verificable con prueba de integración).

• La tabla historial_atencion rechaza operaciones UPDATE/DELETE a nivel de base de

datos. • Los tiempos de respuesta de las operaciones estándar son inferiores a 3000 ms.

• Los contratos JSON usan camelCase y las columnas de base de datos usan snake_case de forma consistente.

2. Modelo de Entidad-Relación

El modelo de datos relacional define la estructura de almacenamiento exclusiva y aislada para el módulo SERVI-X. Para asegurar la escalabilidad dentro del ecosistema de microservicios, el esquema utiliza identificadores universales (UUID v4) como llaves primarias, previniendo colisiones de datos en la red, y aplica estrictamente el estándar snake_case. El modelo garantiza la integridad de la información mediante las siguientes tres entidades principales y sus relaciones de cardinalidad:

Caso (Entidad Central): Es el núcleo transaccional de la PQRS. Almacena los atributos descriptivos (tipo de solicitud, estado) y las referencias externas (cliente, pedido, despacho).

Historial_Atencion (Trazabilidad y Auditoría): Bitácora inmutable diseñada para registrar cualquier transición o actuación sobre una solicitud, almacenando el estado anterior, el nuevo y la respuesta dada.

- **Cardinalidad Caso (1) a Historial (N):** Una relación de uno a muchos que garantiza que un solo caso puede acumular múltiples actuaciones a lo largo del tiempo, pero cada registro histórico pertenece única y exclusivamente a un caso específico.

Responsable (Gestión de Actores): Entidad paramétrica que administra los datos de los agentes internos (nombre, rol, email, estado).

- **Cardinalidad Responsable (1) a Caso (N):** Un agente puede tener asignados múltiples casos simultáneamente bajo su responsabilidad, mientras que un caso, en un momento dado, sólo tiene un único responsable a cargo para evitar conflictos operativos.

- **Cardinalidad Responsable (1) a Historial (N):** Un agente puede generar múltiples registros de actualización en el tiempo. Esta conexión es vital para la auditoría, ya que permite identificar de forma inequívoca qué empleado específico ejecutó cada acción o cambio de estado en el sistema.

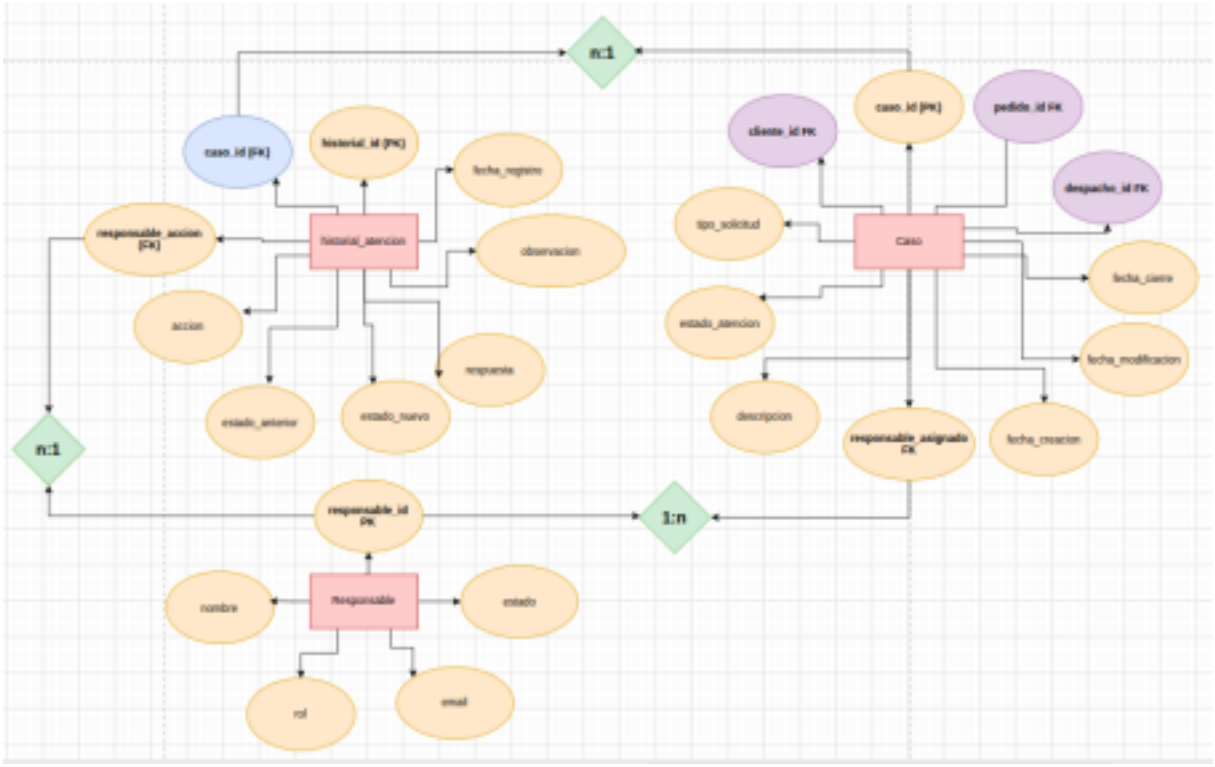


Figura 1. Modelo Entidad-Relación de SERVI-X.

3. Modelo de Casos de Uso

A continuación, se define el comportamiento lógico, las condiciones y el flujo de los casos de uso representados en el modelo UML para el módulo SERVI-X:

ID	Nombre del Caso	Actor	Descripción y Flujo Principal	Precondición
UC-01	Crear usuario	Cliente	Permite el registro de nuevos clientes en el sistema. El flujo incluye ingresar nombre, seleccionar tipo de documento, ingresar número de documento, ingresar email y establecer una contraseña.	Ninguna.
UC-02	Ingresar	Cliente, Agente de servicio	Permite la autenticación de usuarios en el sistema. El flujo incluye digitar el nombre de usuario y la contraseña para obtener acceso según el rol.	Haber creado un usuario previamente.
UC-03	Consultar solicitudes	Cliente, Agente de servicio	Habilita la visualización del listado de PQRS. Permite ingresar número de solicitud y filtrar por fecha, estado o tipo de solicitud.	El usuario debe estar autenticado.
UC-04	Consultar historial	Cliente, Agente de servicio	Muestra la línea de tiempo de un caso. Permite filtrar por fecha, estado, tipo y ver los detalles específicos del historial de atención.	Conocer el ID del caso y estar autenticado.
UC-05	Registrar PQRS	Cliente, Agente de servicio	Permite crear una nueva solicitud. El flujo incluye registrar el tipo específico (Petición, Queja, Reclamo, Solicitud, Felicitación, Observación) y permite registrar pedidos o despachos asociados.	Estar autenticado en el sistema.
UC-06	Asignar responsables	Agente de servicio	Permite seleccionar un caso y asignarle un agente operativo. El flujo incluye elegir el número de solicitud y elegir el responsable.	El caso debe estar en estado “Radicado” o

				“Abierto”.
UC-07	Actualizar estado	Agente de servicio	Modifica la etapa del caso en progreso. El flujo exige ingresar el número de solicitud y seleccionar el nuevo estado, guardando la trazabilidad.	El caso debe estar asignado a un responsable.
UC-08	Generar reportes	Agente de servicio	Consolida métricas operativas del sistema. Permite generar reportes filtrando por fecha, por estado de atención y por tipo de solicitud.	Tener historial registrado en el sistema.
UC-09	Cerrar casos	Agente de servicio	Finaliza el ciclo de vida de la PQRS. El flujo incluye ingresar el número de caso e ingresar una descripción de cierre.	El caso debe tener una respuesta registrada.
UC-10	Registrar respuestas	Agente de servicio	Permite redactar la respuesta formal al cliente. El flujo incluye ingresar el número de caso, el email del cliente y la descripción de la respuesta.	El caso debe estar en proceso de investigación.



Figura 2. Modelo de Casos de Uso de SERVI-X.

3.1 Historias de Usuario

HU-01: Crear usuario

Como Cliente
Quiero registrarme en el sistema SERVI-X
Para poder radicar PQRS y consultar el estado de mis solicitudes.

Criterios de aceptación:

• El formulario en React debe solicitar los campos: nombre completo, tipo de documento, número de documento, correo electrónico y contraseña.

• El sistema debe validar que el correo electrónico no se encuentre previamente registrado en la base de datos PostgreSQL aislada de SERVI-X.

• La contraseña debe almacenarse encriptada mediante algoritmo de hash (ej. bcrypt) antes de persistirse en la base de datos.

• El backend en TypeScript debe generar automáticamente un identificador único bajo el estándar UUID v4 para el nuevo cliente.

• El registro debe persistirse estrictamente en la base de datos relacional PostgreSQL del módulo SERVI-X, sin afectar otros módulos de INTEGRAX.

• La interfaz web debe validar en tiempo real el formato del correo y la fortaleza mínima de la contraseña.

• La petición HTTP debe procesarse y retornar un código de éxito en formato JSON en un tiempo inferior a 3000 ms (RNF-02).

HU-02: Ingresar

Como Cliente o Agente de servicio
Quiero autenticarme en el sistema SERVI-X
Para acceder a las funcionalidades correspondientes según mi rol operativo.

Criterios de aceptación:

• El sistema debe validar las credenciales (usuario y contraseña) contra la base de datos PostgreSQL aislada del módulo.

• El backend en Node.js debe generar un JSON Web Token (JWT) al autenticar exitosamente al usuario.

• El payload del JWT debe incluir el rol del usuario (Cliente vs. Agente) para el control de acceso a las funcionalidades (RNF-04).

• Si las credenciales son incorrectas, la API REST debe retornar un código HTTP 401 sin revelar detalles específicos del error.
• El frontend en React debe almacenar el token de forma segura y redirigir al usuario a su panel correspondiente (Cliente o Agente).

• La respuesta del servidor no debe exceder los 3000 ms de latencia (RNF-02).

HU-03: Consultar solicitudes

Como Cliente o Agente de servicio
Quiero consultar el listado de PQRS registradas en el sistema
Para visualizar el estado actual de las solicitudes y su información general.

Criterios de aceptación:

• El sistema debe retornar el listado estructurado en formato JSON utilizando estrictamente la nomenclatura camelCase para los contratos de red.

• Si el rol autenticado es Cliente, el backend debe filtrar la consulta SQL para retornar únicamente los registros asociados a su UUID.

• La interfaz debe permitir ingresar el número de solicitud para búsqueda directa. • El

listado debe proporcionar filtros funcionales por fecha, estado de atención y tipo de

solicitud.

• La vista desarrollada en React debe ser responsiva y renderizarse correctamente en computadoras de escritorio y dispositivos móviles (RNF-07).

• Los tiempos de respuesta de la consulta no deben exceder los 3 segundos de latencia (RNF-02).

HU-04: Consultar historial

Como Cliente o Agente de servicio
Quiero visualizar la bitácora completa de un caso específico
Para auditar la línea de tiempo y todas las transiciones de estado ocurridas.

Criterios de aceptación:

• El endpoint REST debe exigir un identificador de caso válido (UUID v4) en la ruta para ejecutar la petición.

• El backend debe ejecutar una consulta de solo lectura (SELECT) dirigida exclusivamente a la tabla Historial_Atencion.

• El JSON de respuesta debe incluir los campos: estadoAnterior, estadoNuevo, accionRealizada, usuarioAccionId y fechaRegistro.

• El frontend en React debe renderizar los datos como una línea de tiempo ordenada cronológicamente.
• El middleware de autenticación (JWT) debe garantizar que un cliente solo pueda leer historiales asociados a sus propios casos (RNF-04).

• El sistema debe permitir filtrar el historial por fecha, estado de atención y tipo de solicitud.

• La interfaz debe incluir una opción de "Ver detalles" que expanda la información completa de cada evento del historial.

HU-05: Registrar PQRS

Como Cliente o Agente de servicio
Quiero registrar una nueva PQRS en el sistema
Para escalar un requerimiento operativo, queja, reclamo, solicitud, felicitación u observación formal dentro del módulo.

Criterios de aceptación:

• El sistema debe generar automáticamente un identificador único para el caso utilizando el estándar UUID v4 en formato de texto.

• El sistema debe consumir de forma síncrona vía API REST al módulo CRM-X para validar la identidad del cliente (integración planificada para fase posterior).

• La interfaz en React debe permitir seleccionar la tipología exacta de la solicitud: Petición, Queja, Reclamo, Solicitud, Felicitación u Observación.

• El sistema debe permitir opcionalmente registrar un pedido asociado (consumiendo API de SALES-X) o un despacho asociado (consumiendo API de LOGISTI-X), validando su existencia cuando dicha integración esté habilitada.

• El registro inicial debe almacenarse estrictamente en la base de datos relacional PostgreSQL aislada del módulo SERVI-X.

• La petición HTTP debe procesarse y retornar un código de éxito en formato JSON en un tiempo

inferior a 3000 ms (RNF-02).

• Se debe insertar simultáneamente un registro inicial en la tabla Historial_Atencion para garantizar la trazabilidad de la radicación.

HU-06: Asignar responsables

Como Agente de servicio
Quiero asignar un empleado responsable a un caso abierto
Para garantizar que la solicitud tenga un encargado para su resolución operativa.

Criterios de aceptación:

• El sistema debe consultar la entidad Responsable alojada en la base de datos PostgreSQL de SERVI-X para listar los agentes activos.

• La interfaz debe permitir seleccionar el número de solicitud y elegir el responsable entre los agentes disponibles.

• El backend debe actualizar el atributo de asignación en la tabla Caso mediante la llave UUID del agente.

• La interfaz web debe prevenir que el usuario asigne múltiples responsables simultáneamente al mismo caso.

• El sistema debe registrar el evento de asignación en la bitácora inmutable de auditoría (Historial_Atencion).

• El acceso a esta funcionalidad debe estar validado mediante JSON Web Tokens (JWT) requiriendo el rol de Agente (RNF-04).

HU-07: Actualizar estado

Como Agente de servicio
Quiero actualizar el estado de un caso en progreso
Para reflejar el avance real de la gestión operativa.

Criterios de aceptación:

• El formulario web debe exigir que el agente ingrese una justificación de texto al modificar el estado. • El flujo debe incluir ingresar el número de solicitud y seleccionar el nuevo estado de atención. • El controlador en TypeScript debe capturar el valor del estado anterior y el estado

nuevo. • El sistema debe actualizar el atributo estadoAtencion en la entidad Caso dentro de

PostgreSQL.

• Se debe insertar una fila transaccional en Historial_Atencion que documente ambos estados para garantizar la auditoría inmutable (RNF-05).

• La actualización exitosa debe notificar a la interfaz cliente sin requerir recargar la página completa.

HU-08: Generar reportes

Como Agente de servicio
Quiero generar reportes de gestión del módulo SERVI-X
Para consolidar métricas operativas que apoyen la toma de decisiones.

Criterios de aceptación:

• El backend debe exponer un endpoint REST específico para la consolidación de métricas de negocio.
• El sistema debe ejecutar consultas de agrupación nativas en PostgreSQL para calcular el volumen total.

• La interfaz debe permitir generar reportes filtrando por fecha, por estado de atención y por tipo de solicitud.

• Las métricas retornadas deben agrupar la cantidad de casos dependiendo de su

estadoAtencion. • Las métricas retornadas deben agrupar la cantidad de casos dependiendo

de su tipoSolicitud.

• El payload JSON debe estar formateado para ser renderizado directamente en gráficas por la interfaz web de React sin requerir procesamiento adicional.

HU-09: Cerrar casos

Como Agente de servicio
Quiero cerrar un caso que ya tiene respuesta registrada
Para finalizar el ciclo de vida de la PQRS y bloquear modificaciones futuras.

Criterios de aceptación:

• El backend en Express.js/NestJS debe rechazar la solicitud de cierre si el caso no tiene respuestas previas asociadas.

• El flujo debe incluir ingresar el número de caso e ingresar una descripción de cierre.

• El sistema debe estampar la fecha y hora exacta del cierre en la entidad principal del caso en PostgreSQL (fechaCierre).

• El sistema debe bloquear cualquier operación de actualización (UPDATE) posterior sobre ese registro de caso.

• Se debe generar una traza final en Historial_Atencion documentando al responsable operativo del cierre. • El estado actualizado debe reflejarse inmediatamente en el módulo de generación de reportes. • La operación debe completarse en un tiempo inferior a 3000 ms (RNF-02).

HU-10: Registrar respuestas

Como Agente de servicio
Quiero redactar una respuesta formal dentro del caso
Para brindar una solución oficial al requerimiento del cliente.

Criterios de aceptación:

• El sistema debe habilitar un área de texto extenso en el panel operativo para redactar la respuesta.
• El flujo debe incluir ingresar el número de caso, el correo electrónico del cliente y la descripción de la respuesta.

• La respuesta ingresada debe quedar asociada lógicamente mediante llave foránea (UUID v4) al caso correspondiente.

• El texto oficial debe persistirse como un evento de solución dentro del historial inmutable para prevenir alteraciones futuras (RNF-05).

• La respuesta debe quedar disponible en modo lectura para el cliente en su panel de consulta.

• El sistema debe validar que exista al menos una respuesta oficial registrada antes de habilitar el cierre del caso.

• La notificación al cliente no debe requerir recargar la página completa en el frontend.

4. Modelo de Clases y Estructura de Objetos

El diagrama de clases ilustra la vista estática del backend para el módulo SERVI-X, definiendo la estructura del código en memoria y el comportamiento de los objetos durante la ejecución. Este diseño detallado dicta cómo se implementará la lógica de negocio a nivel granular, respetando los principios de la Programación Orientada a Objetos (POO) y garantizando la interoperabilidad en la red.

Estándares de Diseño y Encapsulamiento

- **Nomenclatura Estandarizada:** Se adoptó estrictamente la notación camelCase para todos los atributos (ej. casoId, fechaRegistro) y métodos de las clases. Esta decisión técnica facilita la serialización y deserialización directa de los objetos hacia estructuras JSON, optimizando el intercambio de datos a través de las APIs REST.

- **Protección de la Información:** Se aplicó el principio de encapsulamiento declarando la totalidad de los atributos con visibilidad privada (indicado por el símbolo -). El estado interno de cada objeto está protegido de alteraciones externas no controladas, y su mutación se realiza exclusivamente con métodos públicos (indicados por el símbolo +).

Definición de Clases y Comportamiento

- **Clase Caso:** Actúa como el objeto central de la lógica de PQRS. Gestiona atributos de negocio esenciales (como tipoSolicitud y estadoAtencion) e invoca métodos de control operativo como actualizarEstado(nuevoEstado) y cerrarCaso().

- **Clase HistorialAtencion:** Representa el objeto transaccional de auditoría. Expone métodos como registrarActualizacion() para garantizar que toda modificación de estado instancie una traza inmutable en la memoria.

- **Clase Responsable:** Modela al actor operativo del sistema. Contiene la información de contacto y métodos administrativos para gestionar el ciclo de vida del agente, como actualizarResponsable().

Asociaciones y Multiplicidad UML

Las relaciones entre las clases se definieron mediante asociaciones estructurales con multiplicidad explícita, reflejando las reglas lógicas del negocio sin mezclar terminología de bases de datos:

- Un objeto Caso está asociado a 1 a muchos (*) objetos de tipo

HistorialAtencion. • Un objeto Responsable administra de 1 a muchos (*)

objetos Caso simultáneamente.

- Un objeto Responsable figura como el autor de 1 a muchos (*) eventos instanciados en el HistorialAtencion.

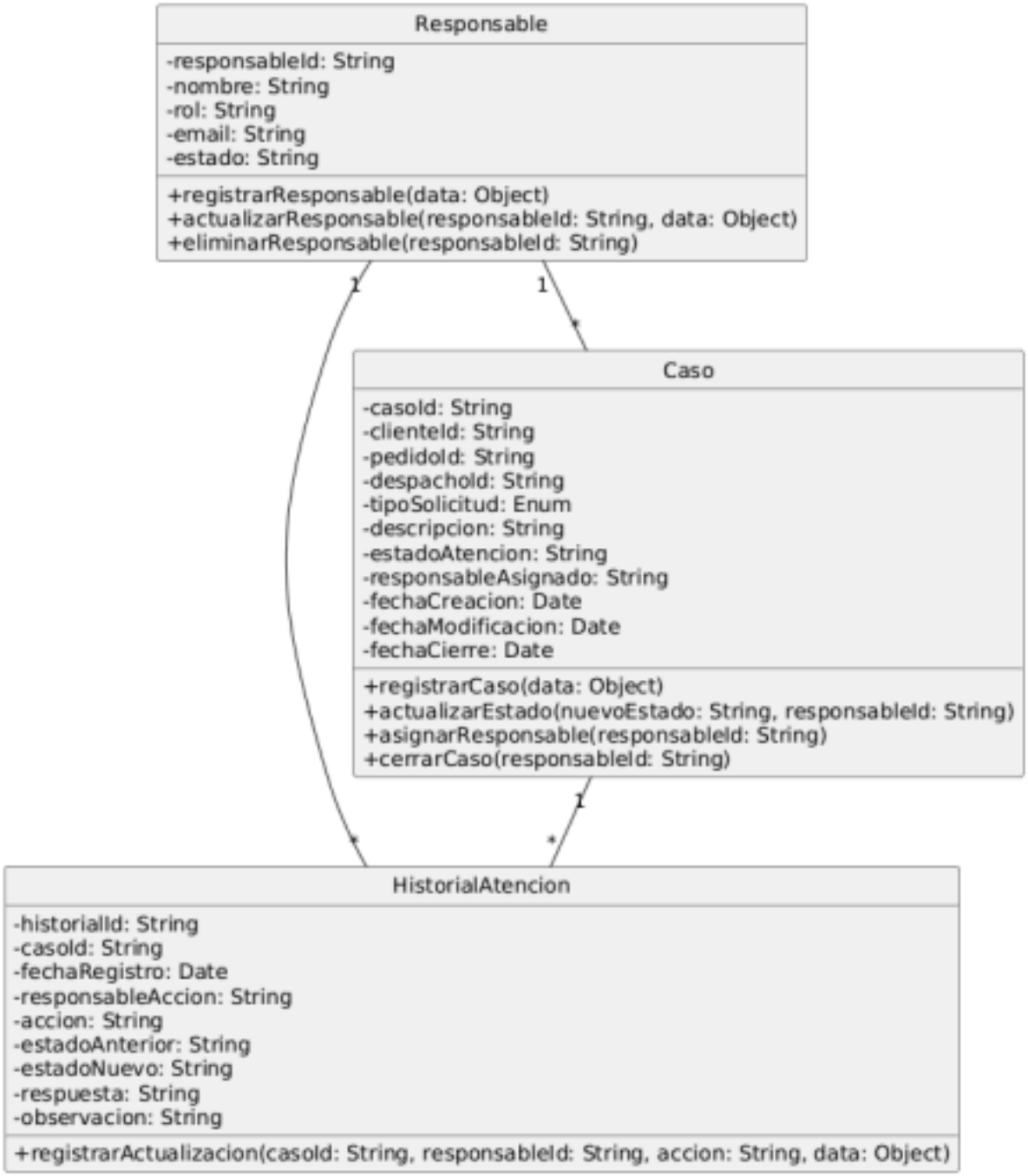


Figura 3. Diagrama de Clases de SERVI-X.

4.1 Diccionario de Datos

Entidad caso

Esta es la tabla transaccional principal que almacena la información base de la radicación. No guarda el detalle de los cambios de estado, ya que esa responsabilidad recae en la auditoría.

Campo	Tipo	Long.	PK	FK	Obligatorio	Descripción y Regla de Negocio
caso_id	UUID	36	Sí	No	Sí	Identificador universal único generado (UUID v4) al radicar.
cliente_id	UUID	36	No	Sí (CRM-X)	Sí	Llave foránea lógica que cruza con el módulo externo CRM-X.
pedido_id	UUID	36	No	Sí (SALES-X)	No	Llave foránea lógica opcional hacia SALES-X (si aplica).
despacho_id	UUID	36	No	Sí (LOGISTI-X)	No	Llave foránea lógica opcional hacia LOGISTI-X (si aplica).
tipo_solicitud	VARCHAR	50	No	No	Sí	Enumeración estricta: 'Peticion', 'Queja', 'Reclamo', 'Sugerencia'.
estado_atencion	VARCHAR	50	No	No	Sí	Estado actual (ej. 'Radicado', 'En Proceso', 'Cerrado').
descripcion	TEXT	Variable	No	No	Sí	Detalle textual de la necesidad o queja expresada por el cliente.
fecha_creacion	TIMESTAMP	N/A	No	No	Sí	Fecha y hora exacta de la radicación (Generada por defecto NOW()).
fecha_cierre	TIMESTAMP	N/A	No	No	No	Se estampa únicamente cuando el caso finaliza su ciclo de vida.
responsable_id	UUID	36	No	Sí (responsable)	No	ID del Agente asignado. FK hacia la tabla responsable.

Entidad historial_atencion

Esta tabla garantiza la trazabilidad absoluta exigida por la arquitectura. Es estrictamente inmutable (solo soporta sentencias INSERT, nunca UPDATE ni DELETE).

Campo	Tipo	Long.	PK	FK	Obligatorio	Descripción y Regla de Negocio
historial_id	UUID	36	Sí	No	Sí	Identificador único del evento registrado (UUID v4).
caso_id	UUID	36	No	Sí (caso)	Sí	Relación obligatoria hacia la tabla caso.

Campo	Tipo	Long.	PK	FK	Obligatorio	Descripción y Regla de Negocio
estado_anterior	VARCHAR	50	No	No	No	Estado previo del caso (Es NULL únicamente en el primer registro de radicación).
estado_nuevo	VARCHAR	50	No	No	Sí	Nuevo estado asignado tras la acción del usuario.
accion_realizada	TEXT	Variable	No	No	Sí	Justificación obligatoria o respuesta oficial brindada.
usuario_accion_id	UUID	36	No	Sí (responsable/cliente)	Sí	ID del usuario (Cliente o Agente) que ejecutó el cambio.
fecha_registro	TIMESTAMP	N/A	No	No	Sí	Estampa de tiempo exacta de la transacción para ordenar la línea de tiempo.

Entidad responsable

Tabla paramétrica para gestionar a los agentes operativos internos autorizados para interactuar con SERVI-X.

Campo	Tipo	Long.	PK	FK	Oblig.	Descripción y Regla de Negocio
responsable_id	UUID	36	Sí	No	Sí	Identificador del agente (UUID v4).
nombre_completo	VARCHAR	150	No	No	Sí	Nombre y apellidos del funcionario.
correo_corporativo	VARCHAR	100	No	No	Sí (único)	Medio de contacto y credencial de acceso.
rol_operativo	VARCHAR	50	No	No	Sí	Nivel de permisos (ej. 'Agente Nivel 1', 'Supervisor').
estado_activo	BOOLEAN	1	No	No	Sí	Indica si el agente está habilitado (TRUE) para recibir asignaciones.

5. Diagrama de Secuencia

El modelo de secuencia expone el comportamiento dinámico del sistema y el flujo temporal de los mensajes intercambiados entre los distintos componentes del módulo SERVI-X. Dado que, en esta fase del proyecto, el módulo opera de manera aislada sin conectarse aún con los servicios externos del ecosistema INTEGRAX (CRM-X, SALES-X y LOGISTI-X), el diagrama ilustra exclusivamente la interacción interna entre el Frontend, la API REST, la capa de Servicio de Negocio y la Base de Datos PostgreSQL aislada.

El diagrama abarca el ciclo de vida completo de una solicitud PQRS, desde la creación del usuario y la autenticación, hasta el cierre del caso, evidenciando las siguientes interacciones clave:

- **Autenticación y Control de Acceso:** Se ilustra el flujo de registro de un nuevo usuario, donde el sistema valida la unicidad del correo, encripta la contraseña mediante algoritmo hash y genera un identificador UUID v4. Posteriormente, el flujo de ingreso muestra la validación de credenciales y la emisión de un JSON Web Token (JWT) que incluye el rol del usuario (Cliente

o Agente), garantizando el control de acceso a las funcionalidades según el RNF-04.

- **Transaccionalidad y Registro de Auditoría:** Se evidencia el comportamiento estricto de persistencia local. Cada vez que el sistema procesa una acción (como la radicación, actualización de estado o cierre de un caso), ejecuta la inserción simultánea en las tablas caso e historial_atencion. Esto garantiza la trazabilidad y la inmutabilidad de la bitácora frente a cualquier cambio de estado, cumpliendo con el RNF-05.

- **Gestión del Ciclo de Vida de la PQRS:** El diagrama recorre las operaciones operativas del agente de servicio: asignación de responsables, clasificación y actualización del estado con justificación obligatoria, registro de la respuesta oficial al cliente, y el cierre definitivo del caso. Este último incorpora un fragmento condicional (alt) que valida la existencia de al menos una respuesta oficial antes de permitir el cierre, bloqueando cualquier modificación futura sobre el registro.

- **Consultas y Reportería:** Se modelan los flujos de consulta de solicitudes con filtros dinámicos, la consulta de la línea de tiempo del historial de atención, y la generación de reportes consolidados mediante consultas de agrupación nativas en PostgreSQL (GROUP BY), retornando un payload JSON listo para ser renderizado en gráficas por el frontend.

- **Estándares de Interoperabilidad:** Todas las respuestas del backend se estructuran en formato JSON utilizando estrictamente la nomenclatura camelCase para los contratos de red, mientras que la persistencia en PostgreSQL emplea snake_case, manteniendo la coherencia con los estándares definidos en el modelado de datos.

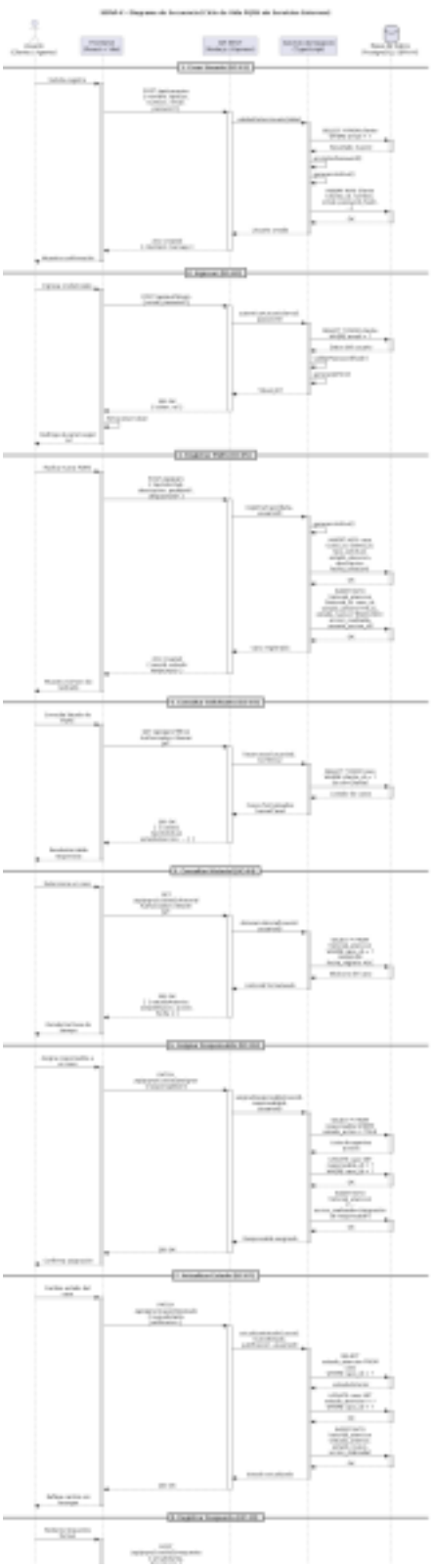


Figura 4. Diagrama de Secuencia - Ciclo de vida de una PQRS (sin servicios externos).

6. Arquitectura

6.1 Ficha técnica de SOFTWARE

Campo	Descripción
Nombre del sistema / proyecto	Módulo SERVI-X (Ecosistema INTEGRAX)
Lenguaje(s) de programación	TypeScript y entorno de Node. Base de datos relacional PostgreSQL
Framework(s) / librerías principales	Express.js o NestJS para exponer la API REST, y React con Vite para consumir los servicios desde el frontend
Motor de base de datos	PostgreSQL configurada como una base de datos relacional
Sistema operativo compatible	Multiplataforma 100% web. Es accesible desde cualquier sistema operativo de escritorio (Windows, macOS, Linux) o móvil (Android, iOS) a través de un navegador web moderno, sin requerir instalaciones locales en el cliente.
Herramientas de desarrollo (IDE, control de versiones, etc.)	Visual Studio Code, Git/GitHub, Docker, Postman, y la metodología (Ágil Scrum / Desarrollo Orientado a Servicios).
Tipo de licenciamiento	Propietario / Académico (Propiedad de INTEGRAX). Ver Matriz de Propiedad Intelectual (sección 7) y Contrato de Licencia (sección 8).
Requerimientos funcionales principales	Gestión del ciclo de vida de PQRS (radicar, clasificar, asignar, actualizar y cerrar casos), registro histórico inmutable y generación de reportes
Módulos / servicios de integración (APIs, TDR asociados)	Consumo planificado (no activo en esta fase) de las APIs REST de los módulos CRM-X (clientes), SALES-X (pedidos) y LOGISTI-X (despachos)

6.2 Ficha técnica de HARDWARE

Campo	Descripción
Tipo de servidor (físico / virtual / nube)	Servidor en la nube virtualizado mediante contenedores Docker y gestionado a través de una plataforma como

	Railway, orquestado para alojar el entorno de Node.js y la base de datos aislada.
Procesador (mínimo / recomendado)	Mínimo 2 vCPU para mantener el contenedor de Node.js y la instancia de PostgreSQL corriendo de forma fluida.
Memoria RAM (mínima / recomendada)	Mínima 2GB / Recomendada 4GB.
Almacenamiento (mínimo / recomendado)	Mínimo 20GB SSD para alojar el sistema operativo del servidor, los módulos de Node y los datos de PostgreSQL.
Requerimientos de red y conectividad	Ancho de banda que garantice que las peticiones HTTP/REST se procesen y retornen una respuesta en un tiempo inferior a 3 segundos.
Dispositivos cliente compatibles	Multiplataforma 100% web: Accesible desde cualquier computadora de escritorio (Windows, macOS, Linux) o dispositivo móvil (Android, iOS).

Campo	**Descripción**
Requerimientos mínimos del cliente/usuario final	Conexión a internet estable y un navegador web moderno (Chrome, Firefox, Safari, Edge) sin necesidad de descargas o instalaciones locales.
Proveedor de infraestructura (on-premise / cloud: AWS, Azure, GCP, etc.)	Despliegue en la nube mediante plataforma como Railway, conectada directamente al repositorio para integración continua y alojamiento de la base de datos PostgreSQL aislada.
Mecanismos de respaldo (backup)	Backups automatizados mediante workflows de GitHub Actions que se disparan periódicamente para generar un dump de la base de datos PostgreSQL y almacenarlo de forma segura en la nube de Cloudflare.

Consideraciones de seguridad física/perimetral	Al estar en la nube, se delega la seguridad física al proveedor (Railway). Perimetralmente, se protegerá el acceso utilizando Cloudflare como proxy inverso y firewall web para mitigar ataques, exponiendo el contenedor de Node.js web únicamente a través de puertos seguros y protegiendo la base de datos.

6.3 Ficha técnica de ARQUITECTURA

Campo	Descripción
Tipo de arquitectura (monolítica, microservicios, SOA, event-driven)	Arquitectura Orientada a Servicios (SOA) con persistencia de datos relacional y aislada
Patrón de integración utilizado	Comunicación síncrona mediante llamadas API REST entre los diferentes módulos del ecosistema (consumo planificado, no activo en esta fase)
Componentes / capas principales del sistema	Capa de Presentación (Frontend Web), Capa de Lógica/API (Controladores construidos en TypeScript/Node.js), y Capa de Datos (PostgreSQL aislada con entidades Caso e Historial)
Protocolos de comunicación (REST, SOAP, gRPC, MQTT, etc.)	Intercambio de mensajes HTTP estrictamente en formato JSON, utilizando nomenclatura camelCase para la red
Middleware / bus de integración (ESB, API Gateway, message broker)	Bus de integración SOA que enrutará las peticiones de red hacia los distintos microproyectos (a habilitar en fase de integración)
Mecanismo de seguridad y autenticación (OAuth2, JWT, API Key, etc.)	Control de acceso basado en JSON Web Tokens y validación de roles operativos (Cliente vs. Agente)
Estrategia de escalabilidad	Uso estricto de identificadores universales (UUID v4) en formato de texto para prevenir colisiones de datos en la red y soportar el crecimiento transaccional

Servicios o APIs externas consumidas	Planificadas (pendientes de habilitar): endpoints síncronos hacia /api/clientes (CRM-X), /api/pedidos (SALES-X) y /api/despachos (LOGISTI-X). En la entrega actual estos datos se gestionan únicamente como llaves foráneas lógicas (cliente_id, pedido_id, despacho_id), sin consumo HTTP activo.

Campo	Descripción
Estrategia de despliegue (CI/CD, contenedores, etc.)	Contenerización del backend de SERVI-X mediante archivos Dockerfile optimizados para entornos Node.js, integrados con despliegue automático desde GitHub hacia Railway

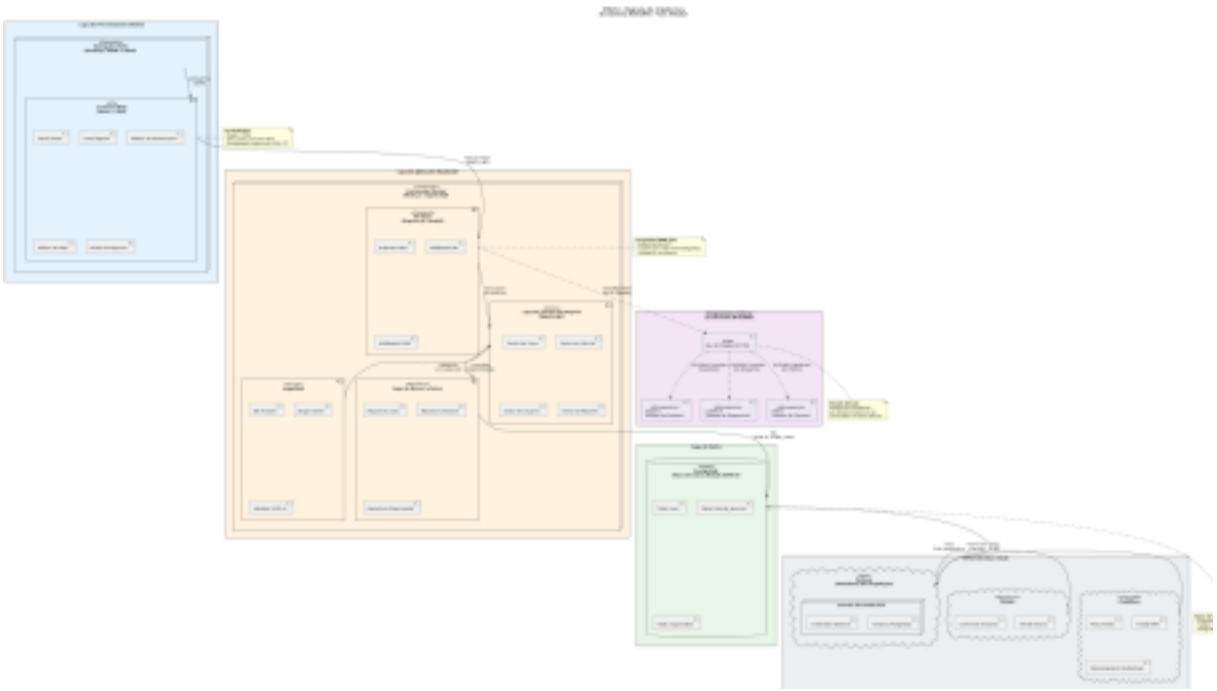


Figura 5. Diagrama de Arquitectura de SERVI-X dentro del ecosistema INTEGRAX.

7. Matriz de Propiedad Intelectual

La siguiente matriz identifica cada componente de software de terceros utilizado en el módulo SERVI-X, su procedencia, su tipo de licencia y el uso específico que se le da dentro del proyecto, diferenciando claramente el código de terceros del código propio del equipo desarrollador.

Elemento	Procedencia	Licencia	Uso en el proyecto

React	Meta Platforms, Inc. / comunidad open source	MIT	Librería base del frontend (SPA, componentes de UI).
Vite	Evan You / comunidad open source	MIT	Bundler y servidor de desarrollo del frontend.
react-router-dom	Remix Software / comunidad open source	MIT	Enrutamiento de vistas en el frontend.
Node.js	OpenJS Foundation	MIT	Entorno de ejecución del backend.
Express.js	OpenJS Foundation / comunidad open source	MIT	Framework web para exponer la API REST.
NestJS (alternativa documentada)	Kamil Mysliwiec / comunidad open source	MIT	Framework alternativo de arquitectura modular para el backend.
TypeScript	Microsoft Corporation	Apache License 2.0	Lenguaje tipado usado en todo el backend.
PostgreSQL	PostgreSQL Global Development Group	Licencia PostgreSQL (estilo MIT/BSD)	Motor de base de datos relacional del módulo.
Prisma ORM (alternativa documentada)	Prisma Data, Inc. / comunidad open source	Apache License 2.0	Mapeo objeto-relacional y migraciones de base de datos.
TypeORM (alternativa documentada)	Comunidad open source	MIT	ORM alternativo basado en decoradores para NestJS.
bcrypt	Comunidad open source	MIT	Hashing de contraseñas de los responsables.
jsonwebtoken	Auth0 / comunidad open	MIT	Emisión y verificación de JSON Web Tokens.

	source		
Jest	OpenJS Foundation / Meta	MIT	Framework de pruebas unitarias e integración del backend.
Supertest	Comunidad open source	MIT	Pruebas de integración sobre los endpoints REST.
Docker	Docker, Inc.	Apache License 2.0 (Docker Engine)	Contenerización del backend y la base de datos para despliegue.

Elemento	Procedencia	Licencia	Uso en el proyecto
Código propio del equipo SERVI-X	Equipo desarrollador (Jose Alvarez Marin)	Derechos reservados - trabajo académico de la Corporación Universitaria de Sabaneta	Lógica de negocio, modelos de dominio, controladores, vistas y configuración específica del módulo SERVI-X.

8. Contrato de Licencia de Software

CONTRATO DE LICENCIAMIENTO Y PRESTACIÓN DE DESARROLLO DE SOFTWARE

Identificación del software: el presente contrato tiene por objeto el software denominado **SERVI-X** (módulo de gestión de PQRS del ecosistema INTEGRAX), en su versión documentada en este mismo documento técnico (modelo de datos, diccionario de datos, modelo de clases y arquitectura descritos en las secciones 2 a 6).

Entre los suscritos, a saber: de una parte, **INTEGRAX** (en adelante, “EL CLIENTE”), plataforma empresarial de integración constituida por los seis microproyectos académicos de la Electiva Profesional II - Integración de Aplicaciones y Sistemas de la Corporación Universitaria de Sabaneta; y de otra parte, el equipo desarrollador del módulo **SERVI-X**, representado por **Jose**

Alvarez Marin (en adelante, "EL DESARROLLADOR"), se celebra el presente contrato académico de licenciamiento de software, regido por las siguientes cláusulas:

CLÁUSULA PRIMERA - Identificación de las partes. EL DESARROLLADOR es el autor y titular original del software denominado SERVI-X, módulo de gestión de PQRS. EL CLIENTE es la plataforma integradora INTEGRAX, que recibe el derecho de uso del módulo para su incorporación dentro del ecosistema empresarial simulado.

CLÁUSULA SEGUNDA - Objeto del contrato. EL DESARROLLADOR concede a EL CLIENTE una licencia de uso sobre el software SERVI-X, en su versión entregada al cierre de cada hito académico (semanas 1 a 5), para su operación, integración y evaluación dentro de la plataforma INTEGRAX.

CLÁUSULA TERCERA - Titularidad. EL DESARROLLADOR conserva en todo momento los derechos morales y la titularidad intelectual sobre el código fuente original de SERVI-X, de conformidad con la Ley 23 de 1982 sobre derechos de autor. La presente licencia no implica cesión ni transferencia de la propiedad intelectual, sino únicamente la concesión de derechos de uso definidos en este contrato.

CLÁUSULA CUARTA - Derechos de uso. EL CLIENTE podrá instalar, ejecutar, consultar e integrar el software SERVI-X con los demás módulos del ecosistema INTEGRAX (CRM-X, SALES-X, FINAN-X, LOGISTI-X, INVENTA-X), exclusivamente dentro del marco académico de la asignatura Integración de Aplicaciones y Sistemas.

CLÁUSULA QUINTA - Duración. El presente contrato tendrá vigencia desde la fecha de su firma hasta la finalización del periodo académico correspondiente (18 de septiembre de 2026), pudiendo prorrogarse por mutuo acuerdo para fases posteriores de integración real.

CLÁUSULA SEXTA - Territorio. La licencia se concede para uso dentro del territorio académico de la Corporación Universitaria de Sabaneta y, de manera extendida, para cualquier entorno de demostración o evaluación docente asociado al proyecto INTEGRAX.

CLÁUSULA SÉPTIMA - Exclusividad. La presente licencia es de carácter no exclusivo. EL DESARROLLADOR conserva el derecho de reutilizar, adaptar o presentar el código fuente de SERVI-X en otros contextos académicos o profesionales, siempre que no comprometa la confidencialidad de datos propios de EL CLIENTE.

CLÁUSULA OCTAVA - Restricciones. EL CLIENTE no podrá sublicenciar, vender, ni distribuir comercialmente el software SERVI-X a terceros ajenos al ecosistema INTEGRAX sin autorización expresa y escrita de EL DESARROLLADOR.

CLÁUSULA NOVENA - Uso comercial. Cualquier explotación comercial del software, fuera del contexto académico aquí descrito, requerirá la negociación de un nuevo acuerdo de licenciamiento entre las partes, incluyendo la correspondiente contraprestación económica.

CLÁUSULA DÉCIMA - Propiedad intelectual. La propiedad intelectual sobre SERVI-X se distribuye de la siguiente manera: (i) el código propio desarrollado por EL DESARROLLADOR permanece bajo sus derechos reservados conforme a la cláusula tercera; (ii) los componentes de terceros incorporados mantienen la titularidad de sus respectivos autores y se rigen por sus licencias originales. EL CLIENTE no adquiere derecho de propiedad alguno sobre ninguno de los dos, únicamente el derecho de uso aquí concedido.

CLÁUSULA DÉCIMA PRIMERA - Componentes de terceros. EL CLIENTE reconoce que SERVI-X incorpora componentes de software de terceros bajo licencias de código abierto (MIT, Apache License 2.0 y Licencia PostgreSQL, entre otras), según se detalla en la Matriz de Propiedad Intelectual (sección 7 de este documento), y se compromete a respetar los términos de dichas licencias.

CLÁUSULA DÉCIMA SEGUNDA - Distribución y modificación. EL CLIENTE no podrá modificar, descompilar, ni generar obras derivadas del código fuente de SERVI-X sin autorización previa y escrita de EL DESARROLLADOR. Cualquier distribución del software a terceros dentro del ecosistema INTEGRAX deberá conservar los avisos de autoría y de licenciamiento originales.

CLÁUSULA DÉCIMA TERCERA - Soporte y mantenimiento. EL DESARROLLADOR prestará soporte y mantenimiento correctivo sobre el software durante la vigencia del periodo académico (cláusula quinta), limitado a la corrección de errores detectados en las pruebas de integración (sección 10 de este documento). Cualquier mantenimiento evolutivo o soporte posterior a la finalización del periodo académico requerirá un acuerdo independiente entre las partes.

CLÁUSULA DÉCIMA CUARTA - Terminación. El presente contrato podrá darse por terminado de forma anticipada por incumplimiento de cualquiera de las partes, o de forma ordinaria al concluir el periodo académico pactado, sin perjuicio de la conservación de los derechos morales de EL DESARROLLADOR.

En constancia de lo anterior, se firma el presente contrato académico el 17 de octubre de 2026.

_____

EL DESARROLLADOR - Jose Alvarez Marin

_____

EL CLIENTE - INTEGRAX

9. Matriz de Trazabilidad

La siguiente matriz demuestra la cadena lógica de trazabilidad entre cada requisito funcional, su caso de uso de origen, la clase y entidad del modelo de datos que lo materializan, el endpoint REST que lo expone y la prueba de integración diseñada para verificarlo.

RF	Caso de Uso (UC)	Clase	Entidad (MER)	Funcionalidad / API (Endpoint REST)	Prueba
RF-01	UC-01 Crear usuario	Responsable (vía Cliente)	caso / responsable	POST /api/v1/usuarios	CP-01
RF-02	UC-02 Ingresar	Responsable	responsable	POST /api/v1/auth/login	CP-02
RF-03	UC-03 Consultar solicitudes	Caso	caso	GET /api/v1/casos	CP-03
RF-04	UC-04 Consultar historial	HistorialAtencion	historial_atencion	GET /api/v1/casos/{id}/historial	CP-04
RF-05	UC-05 Registrar PQRS	Caso	caso	POST /api/v1/casos	CP-05
RF-06	UC-06 Asignar responsables	Caso	caso	PATCH /api/v1/casos/{id}/asignacion	CP-06
RF-07	UC-07 Actualizar estado	Caso	caso	PATCH /api/v1/casos/{id}/estado	CP-07

RF-08	UC-08 Generar reportes	Caso	caso	GET /api/v1/reportes/resumen	CP-08
RF-09	UC-09 Cerrar casos	Caso	caso	PATCH /api/v1/casos/{id}/cierre	CP-09
RF-10	UC-10 Registrar respuestas	HistorialAtencio n	historial_atenci on	POST /api/v1/casos/{id}/respuesta	CP-10

10. Pruebas de Integración